COMPTE RENDU DE PROJET

Billetterie de concerts

Qualité du développement

Réalisé par : Eunice LIGEIRO | Wendy MECHISSO

Enseignant(e) : Oliver CASPARY

Année universitaire : 2026–2027

1. Introduction et objectifs du projet

Dans le cadre de la ressource R5.08 « Qualité du développement », nous avons travaillé sur une application de billetterie de concerts développée avec Java 25, Spring Boot 4.1.1 et MySQL. L'objectif était de construire une API permettant de consulter des concerts, d'inscrire des clients, de gérer une carte de fidélité et d'acheter des billets sans dépasser la capacité des salles. Nous devions également utiliser JPA, des DTO sous forme de records, Flyway, Actuator, Spring Security, Docker et une interface Thymeleaf.

Nous avons surtout cherché à rendre visible notre manière de travailler. Ce compte rendu suit donc les étapes de notre progression : comment nous avons réparti les tâches, les solutions retenues, les erreurs rencontrées, le rôle de l'IA et les vérifications réellement réalisées. Nous distinguons les éléments déjà vérifiés dans une branche de ceux qui demandent encore une intégration finale.

2. Notre organisation et la méthode de travail

2.1. Une répartition pensée pour avancer en parallèle

Nous avons commencé par définir des conventions communes dans CONVENTIONS.md : noms des classes en anglais, tables SQL en snake_case, identifiants Long, DTO records, architecture en couches et contrat des endpoints en français. Cela évitait de prendre des décisions incompatibles à mesure que nous avancions.

Wendy a pris en charge l'infrastructure, le catalogue des artistes, des salles et des concerts, ainsi que les étapes de sécurité. Eunice a développé l'inscription des clients, les cartes de fidélité, la logique de vente et les formulaires Thymeleaf. Cette répartition nous a permis de travailler en même temps, tout en gardant une base technique cohérente.

Nous avons utilisé des branches Git séparées : feature/infrastructure, feature/catalog-database, feature/catalog-api, feature/security et personne-b, avec main destinée au code intégré. Les Pull Requests servent à relire et réunir les changements. Une branch fonctionnelle ne représente pas, à elle seule, une application intégrée et terminée.

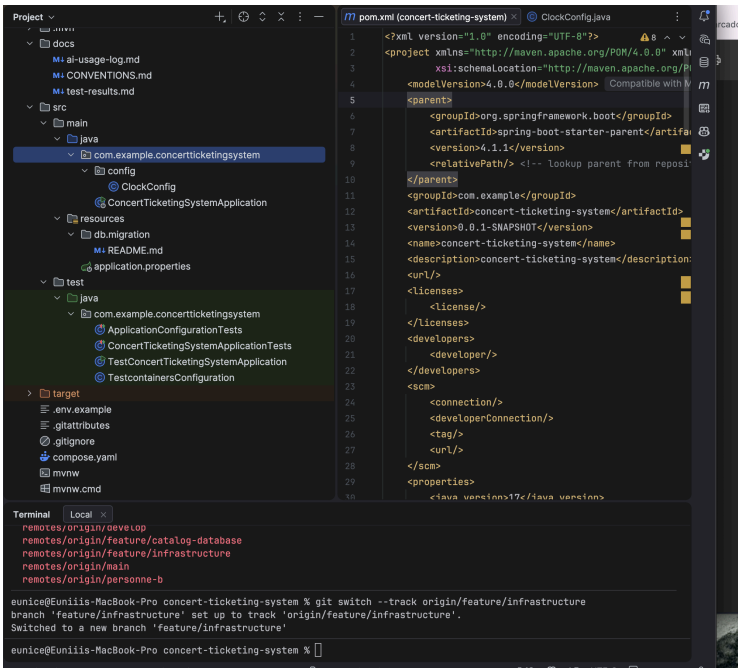


Figure 1 — Consultation de la branche d'infrastructure dans IntelliJ et récupération des branches distantes.

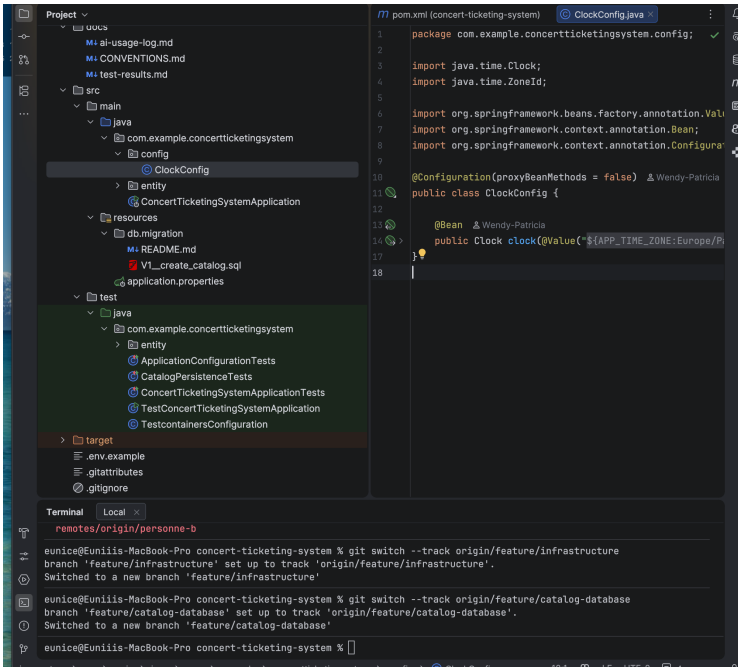


Figure 2 — Consultation de la branche catalogue, distincte de la branche billetterie.

2.2. Comment nous avons limité les dépendances

Une difficulté est apparue rapidement : la vente de billets a besoin du prix d'un concert et de la capacité de sa salle, alors que le catalogue n'était pas encore disponible dans notre branche. Nous avons donc défini une interface ConcertAvailabilityGateway qui décrit exactement les informations utiles. Cela permet de tester les calculs avec Mockito, mais nous savons qu'une

interface seule ne verrouille aucune ligne dans MySQL. La vérification réelle des achats concurrents doit se faire après l'intégration au repository du catalogue.

Nous avons également réservé les versions Flyway : V1 pour le catalogue, V2 pour les clients et les achats, puis V3 pour les données de démonstration. Comme V2 référence concerts, cet ordre est nécessaire.

3. Préparation de l'environnement et choix techniques

3.1. Maven, Java 25 et Docker MySQL

Nous avons créé un projet Maven avec le wrapper versionné, afin d'utiliser les mêmes commandes hors de l'IDE. Le pom.xml a été vérifié et la propriété Java a été alignée sur la version 25 demandée. Pour la base, nous avons privilégié MySQL 8.4 dans Docker Compose : cette solution permet de reproduire l'environnement, de conserver les données dans un volume et de contrôler la disponibilité du conteneur.

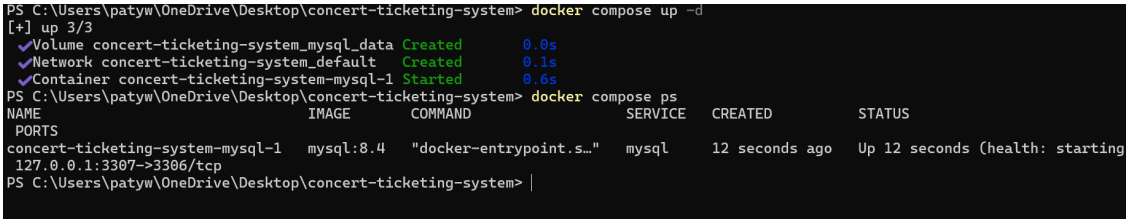


Figure 3 — Démarrage du conteneur MySQL par Docker Compose et contrôle de son état.

La configuration Spring Boot utilise des variables d'environnement pour les identifiants de base, ce qui évite de publier des secrets dans le dépôt. Nous avons relevé un point pratique : le fichier .env utilisé par Compose n'est pas automatiquement chargé dans le processus Java lancé depuis IntelliJ. Les variables doivent donc aussi être transmises à l'application.

3.2. Flyway, JPA et règles de validation

Nous avons conservé spring.jpa.hibernate.ddl-auto=validate : Hibernate vérifie les tables mais ne les crée pas. Les scripts Flyway restent ainsi la seule référence pour la création du schéma. Cette décision nous a parfois ralentis pendant les tests, mais elle évite de masquer une différence entre le code JPA et la base réelle.

```
mysql> SELECT version, description, success
    -> FROM flyway_schema_history;
+---------+----------------+---------+
| version | description    | success |
+---------+----------------+---------+
| 1       | create catalog |       1 |
+---------+----------------+---------+
1 row in set (0.00 sec)
```

Figure 4 — Historique Flyway : la migration V1 « create catalog » est appliquée avec succès.

Pour le catalogue, les entités Artist, Venue et Concert reprennent les contraintes du sujet : nom d'artiste unique, capacité de salle entre 50 et 50 000, date de concert future et prix positif. Les relations avec l'artiste et la salle sont obligatoires. Les montants sont représentés avec BigDecimal, avec un type DECIMAL(10,2) en SQL.

```
mysql> INSERT INTO artists (name, musical_genre)
    -> VALUES ('TestArtistUnique', 'Pop');
Query OK, 1 row affected (0.03 sec)

mysql>
mysql> INSERT INTO artists (name, musical_genre)
    -> VALUES ('TestArtistUnique', 'Rock');
ERROR 1062 (23000): Duplicate entry 'TestArtistUnique' for key '
artists.uk_artists_name'
mysql>
```

Figure 5 — Essai de doublon sur artists.name : MySQL refuse la deuxième insertion.

```
mysql> INSERT INTO venues (name, city, capacity)
    -> VALUES ('TestVenue49', 'Nancy', 49);
ERROR 3819 (HY000): Check constraint 'ck_venues_capacity' is vio
lated.
mysql>
mysql> INSERT INTO venues (name, city, capacity)
    -> VALUES ('TestVenue50001', 'Nancy', 50001);
ERROR 3819 (HY000): Check constraint 'ck_venues_capacity' is vio
lated.
mysql>
mysql> INSERT INTO venues (name, city, capacity)
    -> VALUES ('TestVenue50', 'Nancy', 50);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO venues (name, city, capacity)
    -> VALUES ('TestVenue50000', 'Nancy', 50000);
Query OK, 1 row affected (0.01 sec)
```

Figure 6 — Vérification des bornes de capacité : 49 et 50 001 refusés ; 50 et 50 000 acceptés.

4. Construction progressive de l'application

4.1. Le catalogue et ses endpoints REST

Une fois les entités et la migration du catalogue en place, nous avons développé des repositories JPA, des services et des contrôleurs REST. Les lectures renvoient des DTO records, et les recherches peuvent être filtrées par dates ou par artiste. Les services gardent les vérifications métier, tandis que les contrôleurs restent centrés sur les entrées et les réponses HTTP.

Nous avons retenu des filtres de dates inclusifs et vérifié le cas particulier d'un concert en toute fin de journée. Cette décision a été adaptée à la précision DATETIME(6) de MySQL, pour ne pas perdre les concerts se déroulant à 23:59:59.999999.

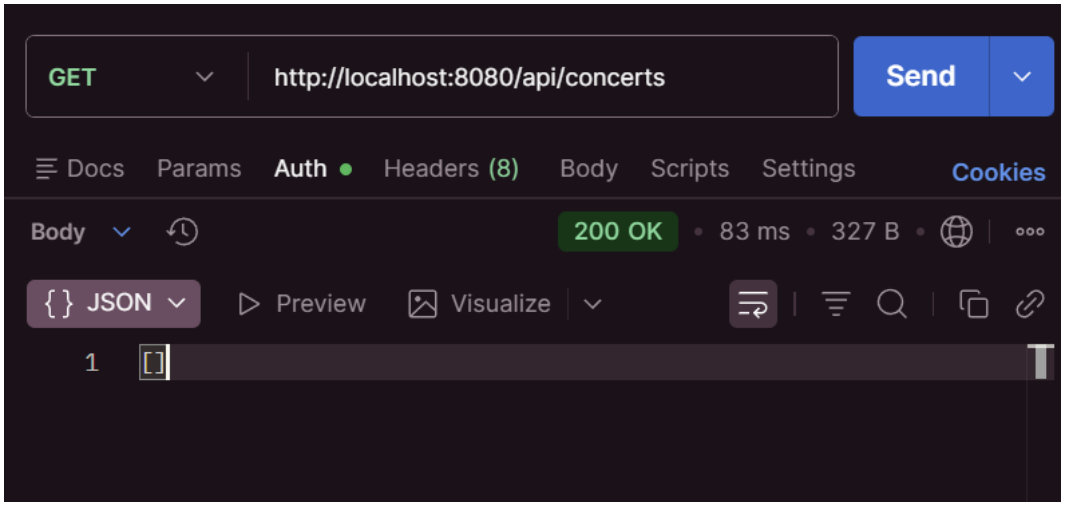


Figure 7 — Lecture du catalogue avec GET /api/concerts : réponse HTTP 200.

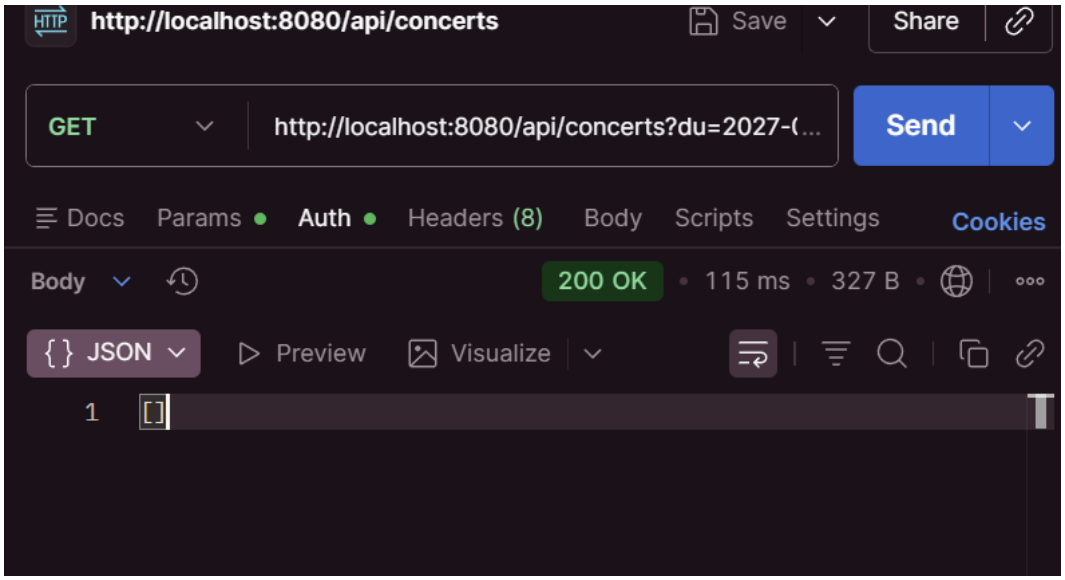


Figure 8 — Recherche de concerts filtrée par période : réponse HTTP 200.

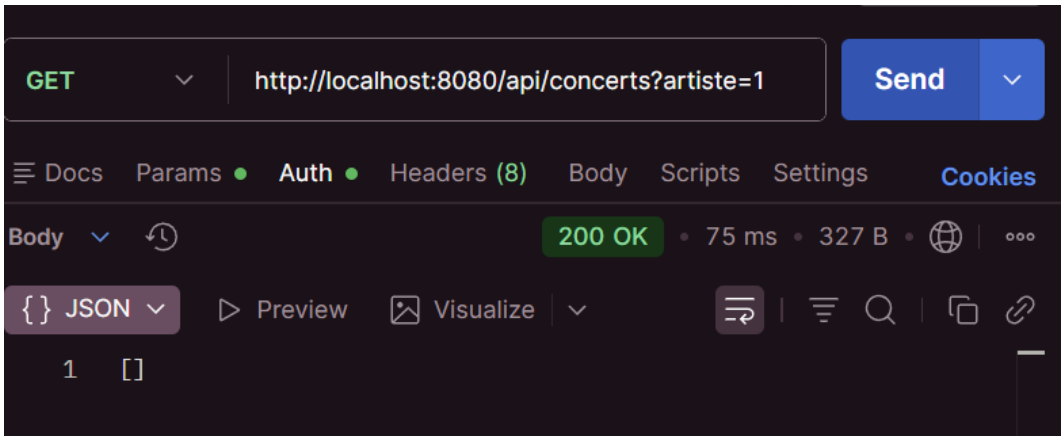


Figure 9 — Recherche de concerts filtrée par artiste : réponse HTTP 200.

4.2. Inscription des clients et fidélité

En parallèle, nous avons construit l'inscription avec Customer, Address, CustomerRepository, CustomerService et CustomerController. L'adresse est intégrée dans Customer avec @Embeddable. Le service vérifie la présence d'un email déjà utilisé, avec une contrainte UNIQUE en base prévue en complément. Les échanges utilisent CustomerRequest et CustomerResponse, sans exposer directement l'entité.

Nous avons ensuite ajouté LoyaltyCard et ses couches associées. La carte est valable un an et un nouveau paiement doit être refusé lorsqu'une carte est encore active. Pour tester correctement les dates, nous avons injecté un Clock : cela permet de fixer le jour du test au lieu de dépendre de la date réelle de la machine.

4.3. Prix des billets, capacité et concurrence

Nous avons isolé les calculs dans TicketPricingService. Cette séparation rend les règles faciles à vérifier : prix unitaire positif, quantité de un à six, remise de 10 % si la carte est active, montant en BigDecimal et arrondi à deux décimales. TicketPurchaseService rassemble ensuite le client, le concert, les places disponibles et l'enregistrement de l'achat.

Dans la branche billetterie, le lien au concert a d'abord été représenté provisoirement par concertId, en attendant la classe Concert. Cette étape a permis d'avancer, mais ne remplace pas le ManyToOne prévu dans CONVENTIONS.md. De même, TicketPurchaseService est annoté @Transactional, mais la prévention réelle de la survente dépend de l'implémentation de findAndLockConcert avec le verrou pessimiste du repository MySQL. Nous ne présentons donc pas le test de concurrence comme réussi à ce stade.

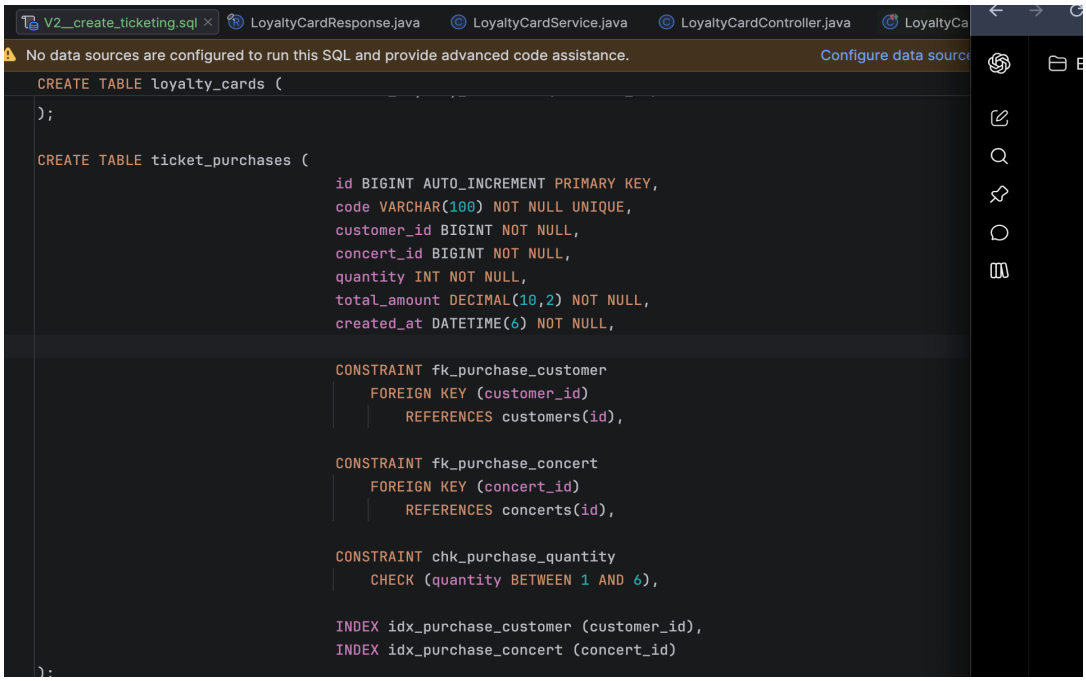


Figure 10 — Préparation de V2__create_ticketing.sql : contraintes et clés étrangères des achats de billets.

4.4. Sécurité et contrôle de santé

Nous avons préparé Spring Security avec HTTP Basic, des mots de passe chiffrés par BCrypt et trois rôles : CLIENT, PRODUCTEUR et ADMIN. Les lectures du catalogue et l'inscription sont publiques ; les opérations de production, d'administration et les achats disposent de règles d'accès différentes. Les erreurs 401 et 403 sont distinguées et les réponses évitent de révéler des détails internes.

Le choix stateless facilite les tests d'API, mais demande de rester attentifs au CSRF pour les pages HTML : une configuration pensée pour Postman et les clients REST ne doit pas être considérée automatiquement comme adaptée aux formulaires Thymeleaf.

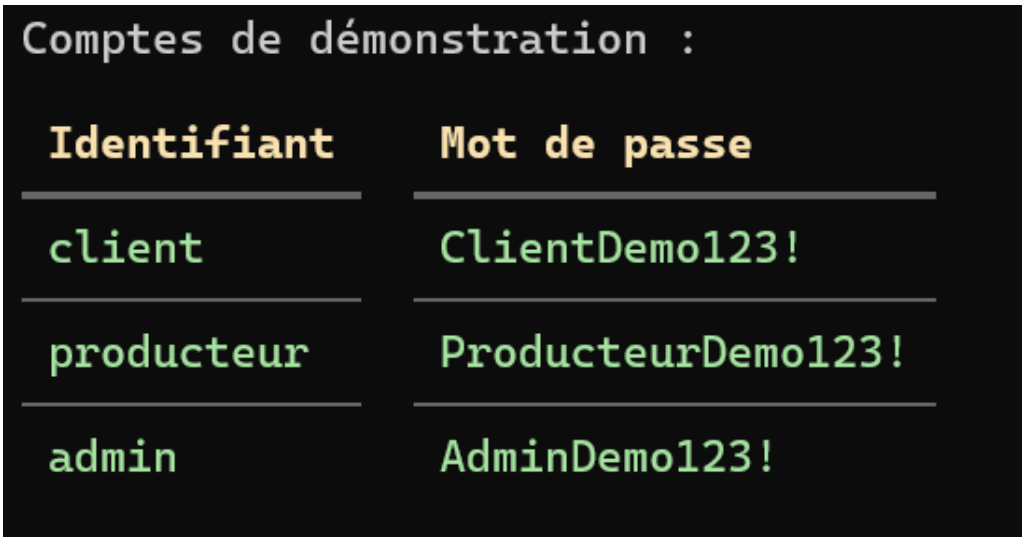


Figure 11 — Comptes de démonstration destinés aux essais des rôles ; identifiants de test, jamais de production.

5. Difficultés rencontrées et décisions prises

Nous avons choisi de garder les erreurs intéressantes plutôt que de ne montrer que les commandes réussies. À chaque fois, nous avons cherché la cause, appliqué une correction ciblée puis relancé une vérification lorsque cela était possible.

Situation	**Analyse / décision**	**Résultat documenté**
./mvnw : permission denied	Le script n'était pas exécutable sur macOS ; correction avec chmod +x mvnw.	Maven a ensuite compilé le code en Java 25.
pom.xml : déclaration XML mal placée / Java 17	Déplacer la déclaration XML au début ; aligner java.version sur 25.	Un clean compile a réussi.
JUnit et Mockito introuvables	CustomerServiceTest avait été placé dans src/main/java ; déplacement vers src/test/java.	Correction effectuée
ConcertAvailabilityGateway.java incohérent	Le fichier contenait la déclaration publique de LoyaltyCardRepository ; remise en place de chaque interface.	Nouvelle compilation à confirmer sur le dernier état.
JAR verrouillé sur Windows	Un ancien processus Java utilisait le JAR ; arrêt ciblé avant clean verify.	Le nettoyage et les vérifications ont ensuite été exécutés.
Quatre assertions SQL CHECK échouent	MySQL 3819 était traduit différemment de l'exception attendue ; test adapté pour vérifier la vraie cause SQL.	47 tests du catalogue passent après correction.
Avertissement SQL « No data sources »	L'éditeur IntelliJ n'avait pas de connexion configurée ; ce n'est pas une exécution Flyway.	Migration V2 rédigée, validation en base à faire.

Ces épisodes montrent aussi les limites des solutions rapides. Par exemple, lorsqu'un test n'arrivait pas à importer Mockito, ajouter des dépendances aurait été inutile : le vrai problème était l'emplacement du fichier. Nous avons donc privilégié la compréhension de l'erreur avant de modifier la configuration.

6. Utilisation raisonnée de l'intelligence artificielle

Nous avons utilisé ChatGPT et Codex comme aides au développement et au diagnostic. Nous n'avons pas voulu présenter leurs réponses comme des preuves. Nous avons comparé les propositions au sujet, au fichier CONVENTIONS.md et aux résultats Maven, puis indiqué les limites restantes. Lorsque le journal mentionne un prompt traduit en français, nous le présentons comme tel et non comme une transcription originale mot pour mot.

6.1. Exemple de prompt et décision — Docker

Prompt de cadrage consigné dans le journal IA (traduction française) : « Créer une configuration Docker Compose pour MySQL. Définir l'image MySQL, le nom du service, le port, la base de données, un volume persistant et un contrôle de santé ; utiliser des variables d'environnement sans publier de secrets. »

Nous avons retenu un service MySQL versionné dans compose.yaml, un volume pour les données et un fichier .env non suivi par Git. Ce choix répondait autant à la reproductibilité qu'à la protection des identifiants.

6.2. Exemple de prompt et décision — Catalogue

Le journal de Codex contient une demande détaillée pour créer Venue et Concert avec @ManyToOne, prix positif, date future et tests MySQL/Testcontainers. L'agent proposait aussi de respecter une migration V1 supposée existante. En inspectant l'espace de travail, il a été constaté que V1 manquait réellement : elle a donc été créée avant les tests. Cet exemple illustre l'importance de vérifier les hypothèses des prompts.

6.3. Exemple de prompt et décision — Erreur de compilation

Après le message Maven « package org.junit.jupiter.api does not exist », ChatGPT a aidé à identifier que le test se trouvait dans src/main/java. Nous avons retenu la correction de l'emplacement et non une modification inutile du POM. Autre exemple : une suggestion de

découplage du catalogue par ConcertAvailabilityGateway a été utile pour les mocks, sans constituer une preuve de verrouillage en base.

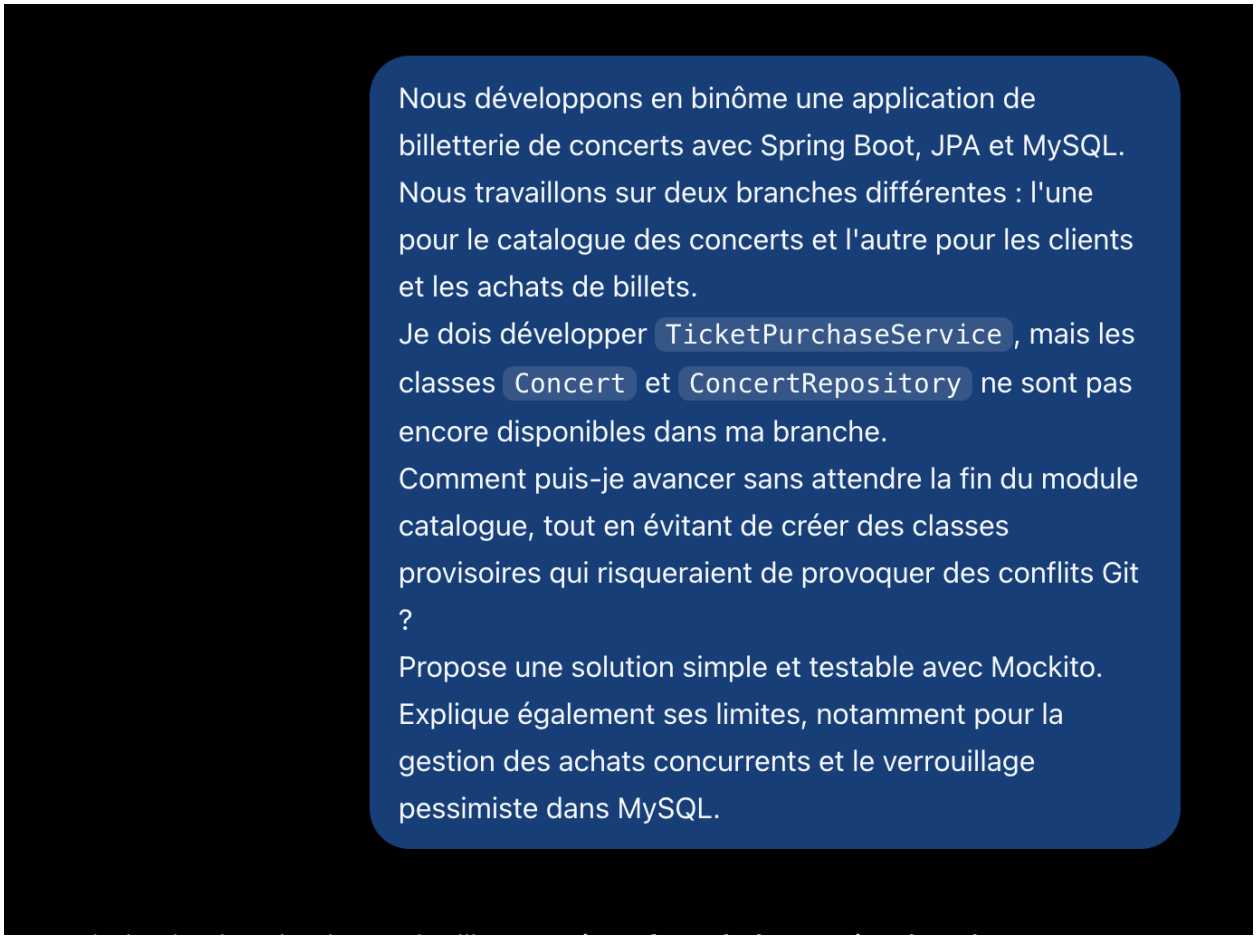


Figure 13 —prompt réellement envoyé à ChatGPT/Codex

7. Intégration, recul critique et conclusion

Notre organisation en branches nous a permis d'avancer sur des fonctionnalités différentes sans attendre la fin de l'ensemble du projet. En contrepartie, nous avons dû être précis sur les contrats partagés, les dépendances de schéma et les responsabilités de chacun. Les vérifications indépendantes de l'infrastructure, du catalogue et de la sécurité constituent une base solide, mais elles ne remplacent pas une exécution de bout en bout.

Le principal enseignement de ce travail est qu'une application ne se résume pas à l'écriture de classes. Nous avons dû comprendre les messages du compilateur, corriger la structure Maven, vérifier les contraintes directement dans MySQL et adapter les assertions à ce que la base renvoyait réellement. Les outils d'IA nous ont fait gagner du temps, à condition de contrôler leurs hypothèses et de ne pas confondre code proposé, code compilé et fonctionnalité validée.

Pour terminer le projet, nous devons réunir les branches, connecter la billetterie aux entités et au verrou réel du catalogue, vérifier V2 puis V3, tester la sécurité des formulaires et exécuter clean verify sur une copie propre du dépôt. Nous compléterons alors ce rapport avec les résultats obtenus, les éventuels conflits de fusion et les dernières captures. Ce compte rendu décrit donc notre démarche et les acquis déjà vérifiés, sans prétendre que les validations encore en attente sont terminées.